

Student Name	CHOWDAM ESWAR	
Student Registration Number	252U1R6052	Class &Section: AIML-2A-203
Study Level : UG/PG	UG	Year &Term: 2 & I
Subject Name	Fundamentals of Artificial Intelligence	

Name of the Assessment	Case Study
Date of Submission	22 – 09 – 2026

Title of Research Paper:	A Survey on NoSQL Stores
Authors:	Ali Davoudian, Liu Chen, and Mengchi Liu
Journal Name & Volume/Issue:	ACM Computing Surveys, Vol. 51, No. 2, Article 40 (43 pages)
Year of Publication:	2018
DOI / URL:	https://doi.org/10.1145/3158661

Date of Review Submission: 22 September 2026

1. Concrete Experience

A. Summary of the Paper

“A Survey on NoSQL Stores” by Davoudian, Chen, and Liu (2018) addresses the difficulties that traditional relational database management systems (RDBMS) face when handling the scale, velocity, and variety of data generated by modern web, mobile, and cloud applications. The authors explain that the rigid, predefined schema of relational

systems makes them costly to evolve, difficult to scale horizontally across commodity servers, and prone to availability problems when distributed ACID transactions are used across many nodes (Davoudian et al., 2018). These limitations gave rise to NoSQL (“Not Only SQL”) stores, a newer generation of database technology designed to give web-scale applications the high availability, horizontal scalability, and flexible, schema-less data handling that RDBMSs struggle to provide.

The purpose of the survey is not to declare NoSQL a replacement for relational databases, but to organize the large and fragmented landscape of NoSQL systems around four fundamental, non-orthogonal design decisions that every distributed data store must make: the data model used to represent information, the consistency model used to coordinate replicas, the strategy used to partition data across nodes, and the trade-offs described by the CAP theorem. For each of these four design dimensions the authors classify the available strategies, describe their strengths and drawbacks, and illustrate them using real academic and industrial systems such as Amazon Dynamo, Google Bigtable, Apache Cassandra, MongoDB, and Neo4j.

The scope of the paper is broad, covering key-value, wide-column, document, and graph data models; consistency models ranging from strong linearizability to eventual and causal consistency; static and workload-aware partitioning strategies; and the CAP/PACELC trade-offs that shape how distributed stores are engineered. The paper closes by identifying open research challenges that the NoSQL community still needs to address, making it a useful reference point for anyone choosing or designing a data store for a large-scale application.

B. Key Findings

- NoSQL stores share common characteristics: schema-less or flexible data models, relaxed (weak) consistency in place of strict ACID guarantees, heavy use of data duplication and “aggregates” to avoid costly joins, and designs that are cluster- and datacenter-friendly for horizontal scaling (Davoudian et al., 2018).
- Four main NoSQL data models exist (key-value, wide-column, document, and graph), each suited to different access patterns; for example, key-value stores fit single-key lookups while graph stores fit deeply connected data.
- Consistency models range from strong guarantees such as linearizability and serializability to weaker models such as eventual consistency, causal consistency, and per-object timeline consistency; weaker models trade strict correctness for lower latency and higher availability.
- Data partitioning can be key-oriented (range-based, simple hashing, consistent hashing, hyperspace hashing) or traversal-oriented for graphs, and can be static or workload-aware to correct load imbalance across nodes.
- The CAP theorem shows that a distributed store can guarantee at most two of Consistency, Availability, and Partition tolerance at once; because network partitions cannot be avoided in practice, most NoSQL stores are designed to sacrifice strict consistency (AP systems) or availability during a partition (CP systems).

- NoSQL stores gain scalability and flexibility at a cost: most lack a standard query language, offer limited support for multi-key transactions, and shift more responsibility for data modeling and consistency handling onto the application developer.

C. Techniques / Research Approach Used

“A Survey on NoSQL Stores” is a survey and taxonomy paper rather than an experimental or empirical study. Davoudian et al. (2018) do not run new experiments or benchmarks of their own; instead, they synthesize and organize existing research and system documentation. Their method is to identify four foundational, non-orthogonal design dimensions of distributed data stores and, for each dimension, build a classification of the strategies used in practice. They then select a representative set of academic systems (e.g., Amazon Dynamo, Scalaris, Spinnaker) and industrial systems (e.g., Cassandra, MongoDB, Neo4j, Google Bigtable) to exemplify each strategy, describing how a real system implements it and what trade-offs result. Throughout, the authors combine textual explanation with conceptual diagrams (such as data-model illustrations and architecture figures) and comparison tables that summarize the strengths, limitations, and representative systems for each category. This qualitative, taxonomy-driven approach suits a fast-moving and fragmented field such as NoSQL, where the goal is to give readers a structured mental model rather than a numerical performance comparison.

D. Figures / Data Found Most Interesting

Because the paper does not present an original experimental dataset, the most useful elements are its classification tables and conceptual diagrams rather than empirical figures. Table 2, “Comparison of NoSQL Data Models,” is especially useful because it lines up the fit scenarios, strengths, and limitations of key-value, wide-column, document, and graph stores side by side, making it easy to reason about which model suits which application. Figure 2, which shows how Facebook’s Inbox Search, McGraw-Hill Education’s learning portal, and Facebook’s social graph actually used the wide-column, document, and graph data models respectively, is also valuable because it grounds the abstract data-model discussion in real production use cases. Table 5, which categorizes representative NoSQL stores according to which CAP guarantee they sacrifice, gives a concrete way to connect the abstract CAP theorem to specific, well-known systems such as Cassandra, MongoDB, and Google Bigtable (Davoudian et al., 2018).

2. Reflective Observation

A. Personal Reflection

The part of the paper I found most interesting was the discussion of the CAP theorem and the more practical PACELC framework it references (Davoudian et al., 2018). Before reading this survey, I understood the CAP theorem only in its simplified textbook form: a system can be consistent, available, or partition tolerant, but not all three at once. What surprised me was learning that “availability” and “consistency” have precise, almost idealized definitions in CAP that do not always match how available or consistent a system feels in practice, and that many real systems, such as Yahoo Pnuts and Google Bigtable, cannot be placed cleanly into a single CAP category because they offer different guarantees for different types of operations.

Reading this survey expanded my understanding of NoSQL databases considerably. Before the reading, I thought of “NoSQL” mainly as “databases that don’t use tables,” associated chiefly with document stores such as MongoDB. The paper made clear that NoSQL is really an umbrella term covering four quite different data models (key-value, wide-column, document, and graph), each with its own strengths, and that the choice of consistency model and partitioning strategy matters just as much as the choice of data model when selecting a store for a given application.

The paper both confirmed and challenged my prior understanding of SQL versus NoSQL. It confirmed my sense that relational databases struggle with horizontal scalability and schema flexibility. However, it challenged the common but oversimplified idea that NoSQL is simply “faster” than SQL: the survey makes clear that NoSQL stores are not universally faster or better; they make different trade-offs, usually sacrificing strong consistency or a standard query language in exchange for availability and scalability. This reframed database selection, for me, as an exercise in matching trade-offs to application requirements rather than picking a technically “superior” system.

The concept I found most difficult to understand was causal consistency, particularly the session guarantees (Monotonic-Reads, Monotonic-Writes, Read-My-Writes, and Writes-Follow-Reads) and how vector clocks are used to track the “happens-before” relationship between operations across replicas. It took me several readings of the messaging-service example in the paper to understand why a causally inconsistent system could let a user read a reply to a message before reading the original message itself.

B. Critical Evaluation

Overall, Davoudian et al. (2018) present a logically organized and well-supported paper. The four-part structure (data model, consistency model, partitioning, and CAP) gives the survey a clear backbone, and each section follows a consistent pattern of defining the concept, classifying the available strategies, and grounding each strategy in specific, cited systems and papers. Nearly every claim is backed by a citation to academic research or official system documentation, which gives the classifications a strong evidentiary basis rather than reading as the authors’ personal opinions.

The paper is also reasonably balanced in its treatment of strengths and weaknesses. For each data model and consistency strategy, the authors explicitly list both benefits (e.g., scalability, easy partitioning) and drawbacks (e.g., limited ad-hoc querying, lack of a standard query language, potential for lost updates). This balance makes the survey more useful as a decision-support tool than as NoSQL advocacy.

That said, the paper has limitations typical of any survey. First, because it synthesizes existing literature rather than running new experiments, it cannot offer independent, quantitative performance comparisons between systems; a reader wanting benchmark numbers, for example Cassandra versus MongoDB, would need to look elsewhere. Second, a survey published in 2018 (submitted in 2016) inevitably misses more recent developments: newer NewSQL and distributed-SQL systems, serverless and multi-model databases, and newer cloud-managed NoSQL offerings are either absent or only briefly acknowledged, since the authors explicitly scope NewSQL and security/privacy models as

outside the survey. Third, because NoSQL technology moves quickly, some of the “representative” system rankings the paper cites (based on tools such as DB-Engines) will already have shifted since publication.

For current database students, the study could be made more useful by pairing its durable conceptual classifications with an accompanying, regularly updated companion resource, for example a maintained comparison table or lab exercises using current versions of the discussed systems, so that the framework the authors provide could be kept current with the fast-changing NoSQL ecosystem.

3. Abstract Conceptualization

A. Related Theories / Models

- Relational database management systems (RDBMS): The paper positions NoSQL as a response to the scalability and schema-rigidity limits of the relational model rather than a wholesale replacement for it (Davoudian et al., 2018).
- NoSQL data models: The survey’s classification of key-value, wide-column, document, and graph models is the conceptual core of the paper and directly maps to how different application data should be structured.
- ACID and BASE properties: The paper contrasts the strict ACID transactions of relational systems with the weaker, higher-availability consistency models (informally summarized elsewhere as BASE: Basically Available, Soft state, Eventually consistent) that many NoSQL stores adopt instead.
- CAP theorem: Section 5 of the paper formalizes the Consistency–Availability–Partition-tolerance trade-off and classifies representative NoSQL stores by which guarantee they sacrifice.
- Horizontal scaling: The paper repeatedly frames NoSQL’s central motivation as enabling systems to scale out across many commodity servers rather than scaling up a single, more powerful machine.
- Sharding and replication: Section 4 of the paper explains horizontal partitioning (sharding) strategies in depth, and Appendix B (referenced in the survey) covers replication protocols used to keep shards available and fault-tolerant.
- Eventual consistency: Section 3.2 explains eventual consistency as the default consistency model for many production NoSQL stores, along with the conflict-resolution and repair strategies (e.g., read-repair, vector clocks) needed to make it work.

B. Contribution to Academic Understanding

The paper contributes to academic understanding by teaching that there is no single “best” database architecture; instead, choosing a database is a structured engineering trade-off. Davoudian et al. (2018) show that an application’s data-access pattern (single-key lookups versus deep relationship traversals), its consistency requirements (strict correctness versus high availability), and its growth pattern (vertical versus horizontal scaling) should all drive the choice between a relational system and a specific category of NoSQL store. This gives students a repeatable decision framework (data model, consistency model, partitioning

strategy, and CAP trade-off) that can be applied to almost any new application rather than requiring memorization of individual products.

C. Concepts Learned

- **Aggregates:** NoSQL stores group related data together (e.g., a whole document or column-family) so that common queries need only touch a single node.
- **Consistent hashing:** A hashing technique that lets nodes be added or removed while redistributing only a small fraction of the data, unlike simple modulo hashing.
- **Vector clocks:** A mechanism for tracking causal “happens-before” relationships between operations across replicas without relying on synchronized physical clocks.
- **Read-repair and hinted-handoff:** Background mechanisms that help eventually consistent stores converge stale replicas back to the latest value.
- **PACELC:** An extension of CAP that also accounts for the trade-off between consistency and latency when there is no network partition.
- **Traversal-oriented partitioning:** A graph-specific partitioning approach that tries to keep highly connected vertices together to minimize costly cross-partition traversals.
- **Atomic aggregates:** A limited form of transaction support, used by some NoSQL stores, that guarantees strict consistency only within a single aggregate (e.g., one document or row) rather than across the whole database.

D. Connection to DBMS Course Topics

- **Database architecture:** The paper’s four-generation history of database technologies (hierarchical, relational, object-oriented, NoSQL/NewSQL) directly extends the architecture history covered in the DBMS course.
- **Data models:** The key-value, wide-column, document, and graph models studied here broaden the relational data model taught as the course’s foundation.
- **Distributed databases:** The paper’s treatment of replication, partitioning, and the CAP theorem applies the course’s single-node concepts (normalization, indexing) to a multi-node, distributed setting.
- **Transactions and consistency:** The comparison of linearizability, serializability, and eventual/causal consistency extends the course’s ACID-transaction material to weaker, more scalable consistency models.
- **Query processing:** The paper’s discussion of the lack of a standard NoSQL query language and of application-side joins highlights why query optimization is handled differently than in SQL-based systems.
- **Scalability and performance:** The paper’s partitioning and sharding strategies connect directly to the course’s discussion of indexing and performance tuning, but at a distributed-systems scale.
- **Database design decisions:** The paper’s emphasis on choosing a data model and consistency level based on application requirements mirrors the course’s schema-design and normalization decisions, just applied to a denormalized, aggregate-oriented context.

4. Active Experimentation

A. Possible Applications

- E-commerce product catalog: A document store (e.g., MongoDB) would suit a product catalog well because product attributes vary widely between categories and the flexible, schema-less document model can store each product's attributes without forcing every item into identical columns.
- Social media application: A graph store (e.g., Neo4j) fits the friend/follow relationships and activity feeds of a social network, since the paper shows that graph traversal queries (such as finding "friends of friends") are handled far more efficiently by graph stores than by relational joins.
- Internet of Things (IoT) sensor data: A wide-column or key-value store (e.g., Cassandra) suits high-volume, high-velocity IoT sensor readings, because these stores are optimized for fast writes and horizontal scaling across many nodes, which the paper identifies as a core NoSQL strength.
- Student or library management system: A hybrid SQL–NoSQL approach fits best here. Core records such as student enrollment, grades, and book loans need strong consistency and multi-table transactions, which favors a relational database, while supplementary data such as library search logs or student activity analytics could use a NoSQL store optimized for flexible, high-volume data.

B. Future Research Ideas

- Benchmarking MongoDB and MySQL using a common workload (e.g., TPC-C-style transactions) to quantitatively compare throughput and latency under different consistency settings.
- Comparing eventual consistency and strong consistency performance under simulated network conditions (latency, packet loss) to observe the practical effect of the CAP trade-off described in the paper.
- Designing a small hybrid SQL–NoSQL architecture for a sample application (e.g., an online bookstore) and documenting which data belongs in which store and why.
- Studying database selection criteria for IoT applications with constrained bandwidth, comparing a wide-column store against a time-series-specialized store.
- Exploring how machine learning could be used for automatic database performance tuning, such as predicting hotspots for the workload-aware partitioning strategies discussed in the paper.

C. Application to My Next Project

For my next DBMS project, I could apply what I learned from this survey by building a "Campus Lost and Found" application. Project title: "Campus Lost and Found Tracker." Database choice: a hybrid architecture, using a relational database (e.g., MySQL or PostgreSQL) for core, strongly consistent records such as user accounts and claim/verification transactions, and a document store (e.g., MongoDB) for the lost-item reports themselves, since each report may include a variable number of photos, tags, and free-text descriptions that do not fit neatly into a fixed relational schema. A possible

document collection, “lost_items,” could store fields such as item_id, reporter_id, category, description, tags (an array), photo_urls (an array), location_found, and status, while the relational side would keep a “users” table and a “claims” table with foreign keys and transactional guarantees to ensure an item cannot be claimed by two people at once. This split follows the paper’s own reasoning: use NoSQL where flexibility and scale matter most, and keep relational guarantees where correctness of a transaction (such as approving a claim) is critical.

D. Personal Action Plan

- I will read the abstract, introduction, and conclusion of a research paper first to build a mental map before working through the detailed sections.
- I will keep a running glossary of unfamiliar terms (such as “linearizability” or “causal consistency”) as I read, instead of skipping past them.
- I will look for the classification tables and figures in a survey paper early, since they often summarize the paper’s main contribution more compactly than the prose.
- I will practice explaining difficult concepts, such as vector clocks, in my own words or with a small diagram to check whether I actually understood them.
- I will consciously note a paper’s publication date and scope limitations so that I do not treat an older survey’s system comparisons as fully current.
- I will apply concepts from papers to a small personal project soon after reading, since building the “Campus Lost and Found” schema above helped solidify my understanding of when to choose NoSQL over SQL.

Summary Table (Optional)

Kolb Stage	Key Insights / Points from Your Review
Concrete Experience	Reviewed a survey of NoSQL stores covering data models, consistency models, partitioning, and CAP; identified key findings, the taxonomy-based research approach, and the most useful classification tables and figures.
Reflective Observation	Reflected on how the paper reframed my understanding of NoSQL as trade-off driven rather than “better” than SQL; identified causal consistency and vector clocks as the most difficult concepts; critically evaluated the paper’s balance and its limitations as a 2018 survey.
Abstract Conceptualization	Connected the paper to RDBMS, ACID/BASE, the CAP theorem, horizontal scaling, sharding/replication, and eventual consistency; linked the

	survey's framework to database architecture, data models, distributed databases, and query-processing topics from the DBMS course.
Active Experimentation	Identified practical applications (e-commerce, social media, IoT, library systems), proposed future research/benchmarking ideas, and designed a hybrid SQL/NoSQL schema for a "Campus Lost and Found" project as a next step.

References

Davoudian, A., Chen, L., & Liu, M. (2018). A survey on NoSQL stores. *ACM Computing Surveys*, 51(2), Article 40. <https://doi.org/10.1145/3158661>